

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT on

Artificial Intelligence (23CS5PCAIN)

Submitted by

Anag Devnani (1BM23CS031)

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Sep-2024 to Jan-2025

B.M.S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “Artificial Intelligence (23CS5PCAIN)” carried out by **Anag Devnani (1BM23CS031)**, who is a bonafide student of **B.M.S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements in respect of an Artificial Intelligence (23CS5PCAIN) work prescribed for the said degree.

Dr. K. R. Mamatha Associate Professor Department of CSE, BMSCE	Dr. Kavitha Soodha Professor & HOD Department of CSE, BMSCE
--	---

Index

Sl. No.	Experiment Title	Page No.
1	Implement Tic –Tac –Toe Game Implement vacuum cleaner agent	
2	Implement 8 puzzle problems using Depth First Search (DFS) Implement Iterative deepening search algorithm	
3	Implement A* search algorithm	
4	Implement Hill Climbing search algorithm to solve N-Queens problem	
5	Simulated Annealing to Solve 8-Queens problem	
6	Create a knowledge base using propositional logic and show that the given query entails the knowledge base or not.	
7	Implement unification in first order logic	
8	Create a knowledge base consisting of first order logic statements and prove the given query using forward reasoning.	
9	Create a knowledge base consisting of first order logic statements and prove the given query using Resolution	
10	Implement Alpha-Beta Pruning.	

Infosys Springboard Certificates



CERTIFICATE OF ACHIEVEMENT



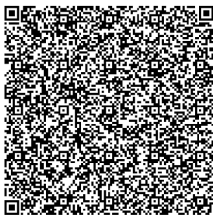
The certificate is awarded to

Anag Devnani

for successfully completing

Artificial Intelligence Foundation Certification

on November 24, 2025



Issued on: Monday, November 24, 2025
To verify, scan the QR code at <https://verify.onwingspan.com>



Congratulations! You make us proud!

Satheesha B.N.
Satheesha B. Nanjappa
Senior Vice President and Head
Education, Training and Assessment
Infosys Limited



COURSE COMPLETION CERTIFICATE



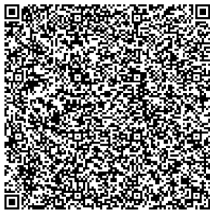
The certificate is awarded to

Anag Devnani

for successfully completing the course

Introduction to Artificial Intelligence

on November 24, 2025



Issued on: Tuesday, November 25, 2025
To verify, scan the QR code at <https://verify.onwingspan.com>



Congratulations! You make us proud!

Satheesha B.N.
Satheesha B. Nanjappa
Senior Vice President and Head
Education, Training and Assessment
Infosys Limited



Github Link:

<https://github.com/AnagDevnani/BMS-Lab/tree/main/AI>

Program 1

Implement Tic –Tac –Toe Game

Implement vacuum cleaner agent

Algorithm:

(You should provide the screen shot of your observation book of Algorithm/Logic/Solving of respective problem)

Code:

```
import math

def print_board(board):
    for row in board:
        print(" | ".join(row))
    print("-" * 5)

def check_winner(board, player):
    # Rows, columns, diagonals
    for row in board:
        if all(cell == player for cell in row):
            return True

    for col in range(3):
        if all(board[row][col] == player for row in range(3)):
            return True

    if all(board[i][i] == player for i in range(3)) or \
        all(board[i][2-i] == player for i in range(3)):
        return True

    return False

def is_full(board):
    return all(cell != " " for row in board for cell in row)

def minimax(board, depth, is_maximizing):
    if check_winner(board, "O"): # Computer wins
        return 1
    if check_winner(board, "X"): # Player wins
        return -1
    if is_full(board):
        return 0

    if is_maximizing: # Computer's move
```

```

        best_score = -math.inf
    for i in range(3):
        for j in range(3):
            if board[i][j] == " ":
                board[i][j] = "O"
                score = minimax(board, depth + 1, False)
                board[i][j] = " "
                best_score = max(score, best_score)
    return best_score
else: # Player's move
    best_score = math.inf
    for i in range(3):
        for j in range(3):
            if board[i][j] == " ":
                board[i][j] = "X"
                score = minimax(board, depth + 1, True)
                board[i][j] = " "
                best_score = min(score, best_score)
    return best_score

def best_move(board):
    best_score = -math.inf
    move = None
    for i in range(3):
        for j in range(3):
            if board[i][j] == " ":
                board[i][j] = "O"
                score = minimax(board, 0, False)
                board[i][j] = " "
                if score > best_score:
                    best_score = score
                    move = (i, j)
    return move

def tic_tac_toe():
    board = [[" " for _ in range(3)] for _ in range(3)]
    print("Welcome to Tic-Tac-Toe! You are 'X' and computer is 'O'.")
    print_board(board)

    while True:
        # Player move
        while True:
            try:
                row = int(input("Enter row (0-2): "))
                col = int(input("Enter col (0-2): "))
                if board[row][col] == " ":
                    board[row][col] = "X"

```

```

        break
    else:
        print("Cell already taken, try again.")
except (ValueError, IndexError):
    print("Invalid input! Enter numbers 0-2.")

print_board(board)

if check_winner(board, "X"):
    print("You win!")
    break
if is_full(board):
    print("It's a draw!")
    break

# Computer move
print("Computer's turn...")
move = best_move(board)
if move:
    board[move[0]][move[1]] = "O"

print_board(board)

if check_winner(board, "O"):
    print("🖥️ Computer wins!")
    break
if is_full(board):
    print("It's a draw!")
    break

if __name__ == "__main__":
    tic_tac_toe()
# vacuum cleaner
def vacuum_cleaner(rooms):
    print("Initial Room Status:")
    print(rooms)

    # Traverse through each room
    for i in range(len(rooms)):
        if rooms[i] == 0:
            print(f"Room {i+1} is Dirty. Cleaning...")
            rooms[i] = 1
        else:
            print(f"Room {i+1} is already Clean.")

    print("\nFinal Room Status:")

```



```
print(rooms)
```

0 = Dirty, 1 = Clean

Example: [0, 1, 0, 1] means Room1=Dirty, Room2=Clean, Room3=Dirty, Room4=Clean

rooms = [0, 1, 0, 1]

```
vacuum_cleaner(rooms)
```

Date: 20/8/25
Page: _____

Tic Tac Toe

Algorithm / Pseudocode:

```
import math;
def evaluate(b):
    for row in range(3):
        if b[row][0] == b[row][1] == b[row][2]: return 10
        if b[0][row] == b[1][row] == b[2][row]: return -10
    return 0

def minimax(b, is_max):
    score = evaluate(b)
    if score > 0: return score
    best = -math.inf
    if is_max else math.inf
    for i in range(3):
        for j in range(3):
            if b[i][j] == ' ':
                b[i][j] = 'O' if is_max else 'X'
                val = minimax(b, not is_max)
                b[i][j] = ' '
                best = max(best, val) if is_max else min(best, val)
    return best

def bestMove(b):
    bestMove = -math.inf, (1, -1)
    for i in range(3):
        for j in range(3):
            if b[i][j] == ' ':
                b[i][j] = 'O'
                score = minimax(b, False)
```

Date: ___/___/___
Page: _____

$b[i][j] = ' '$ if score > best:
bestMove = row(i, j)
return move

O/P:

X		
	O	

X		
	O	O

X → player
O → AI

X		
O	O	O
X		X

(e) AI wins.

Date 20/8/25
Page _____

Vacuum cleaner

```

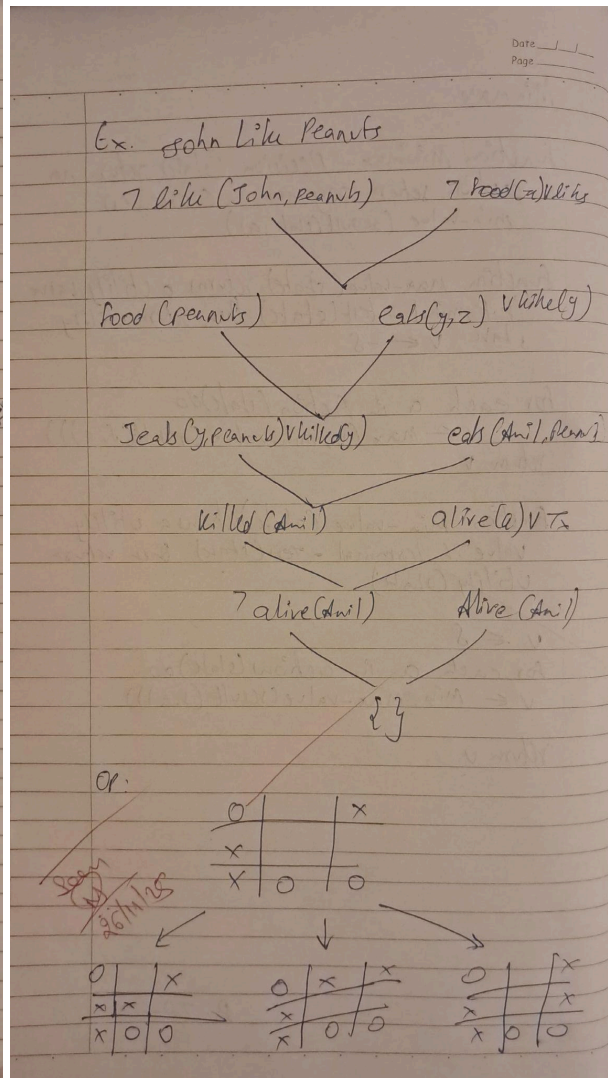
room = {
    'A': 'Dirty',
    'B': 'Dirty'
}

vac_loc = 'A'

def suck():
    print("Sucking dirt in room {vac_loc}")
    rooms[vac_loc] = 'clean'

def move():
    global vac_loc
    if (vac_loc == 'A'):
        print("move to B")
        vac_loc = 'B'
    else if (vac_loc == 'B'):
        print("move to C")
        vac_loc = 'C'
    else:
        print("to A")
        vac_loc = 'A'

while dirty in room.values():
    if rooms[vac_loc] == 'Dirty':
        suck()
    else:
        move()
  
```



Program 2

Implement 8 puzzle problems using Depth First Search (DFS)

Implement Iterative deepening search algorithm

Algorithm:

(You should provide the screen shot of your observation book of Algorithm/Logic/Solving of respective problem)

Code:

```
import heapq
from itertools import count
from typing import Callable, Dict, Iterable, List, Optional, Tuple
from functools import lru_cache

State = Tuple[int, ...] # 9-length tuple; 0 represents the blank
GOAL: State = (1, 2, 3, 4, 5, 6, 7, 8, 0)

# Directions for blank movement: (dr, dc, move_char)
DIRS = [(-1, 0, 'U'), (1, 0, 'D'), (0, -1, 'L'), (0, 1, 'R')]

def index_to_rc(i: int) -> Tuple[int, int]:
    return divmod(i, 3)

def rc_to_index(r: int, c: int) -> int:
    return r * 3 + c

def is_solvable(state: State) -> bool:
    """An 8-puzzle is solvable iff the inversion count (ignoring 0) is even."""
    arr = [x for x in state if x != 0]
    inv = 0
    for i in range(len(arr)):
        for j in range(i + 1, len(arr)):
            if arr[i] > arr[j]:
                inv += 1
    return inv % 2 == 0

def neighbors(state: State) -> Iterable[Tuple[State, str]]:
    """Yield (neighbor_state, move_char) for each legal slide."""
    i0 = state.index(0)
    r0, c0 = index_to_rc(i0)
    for dr, dc, mv in DIRS:
        nr, nc = r0 + dr, c0 + dc
        if 0 <= nr < 3 and 0 <= nc < 3:
```

```

        ni = rc_to_index(nr, nc)
        new_state = list(state)
        new_state[i0], new_state[ni] = new_state[ni], new_state[i0]
        yield tuple(new_state), mv

def reconstruct_path(parents: Dict[State, State], end: State) -> List[State]:
    path = [end]
    while end in parents:
        end = parents[end]
        path.append(end)
    path.reverse()
    return path

def path_to_moves(path: List[State]) -> str:
    """Convert a path of states into a string of moves 'UDLR'."""
    moves = []
    for a, b in zip(path, path[1:]):
        ia, ib = a.index(0), b.index(0)
        ra, ca = index_to_rc(ia)
        rb, cb = index_to_rc(ib)
        if rb == ra - 1 and cb == ca:
            moves.append('U')
        elif rb == ra + 1 and cb == ca:
            moves.append('D')
        elif rb == ra and cb == ca - 1:
            moves.append('L')
        elif rb == ra and cb == ca + 1:
            moves.append('R')
        else:
            moves.append('?')
    return "".join(moves)

def misplaced_tiles(state: State, goal: State = GOAL) -> int:
    """Heuristic: number of tiles not in their goal position (ignore blank)."""
    return sum(1 for i, v in enumerate(state) if v != 0 and v != goal[i])

@lru_cache(maxsize=None)
def _goal_positions(goal: State) -> Tuple[Tuple[int, int], ...]:
    """Cache goal positions as a tuple indexed by tile value (0..8)."""
    pos = [(0, 0)] * 9
    for i, v in enumerate(goal):
        r, c = index_to_rc(i)
        pos[v] = (r, c)

```

```

return tuple(pos)

def manhattan_distance(state: State, goal: State = GOAL) -> int:
    """Heuristic: sum of Manhattan distances of each tile to its goal position."""
    pos = _goal_positions(goal)
    total = 0
    for i, v in enumerate(state):
        if v == 0:
            continue
        r, c = index_to_rc(i)
        gr, gc = pos[v]
        total += abs(r - gr) + abs(c - gc)
    return total

def a_star(
    start: State,
    heuristic: Callable[[State, State], int] = manhattan_distance,
    goal: State = GOAL,
) -> Optional[Dict[str, object]]:
    """
    A* search for the 8-puzzle.
    Returns a dict with keys: path, moves, expanded_nodes, or None if unsolvable.
    """
    if start == goal:
        return {"path": [start], "moves": 0, "expanded_nodes": 0}

    if not is_solvable(start):
        return None

    # Each heap item: (f, g, tie, state)
    heap: List[Tuple[int, int, int, State]] = []
    tie = count()
    g0 = 0
    h0 = heuristic(start, goal)
    heapq.heappush(heap, (g0 + h0, g0, next(tie), start))

    best_g: Dict[State, int] = {start: 0}
    parents: Dict[State, State] = {}
    expanded = 0

    while heap:
        f, g, _, state = heapq.heappop(heap)
        expanded += 1

        if state == goal:

```

```

    path = reconstruct_path(parents, state)
    return {
        "path": path,
        "moves": len(path) - 1,
        "expanded_nodes": expanded,
        "move_sequence": path_to_moves(path),
    }

    for nb, _mv in neighbors(state):
        new_g = g + 1
        if nb not in best_g or new_g < best_g[nb]:
            best_g[nb] = new_g
            parents[nb] = state
            h = heuristic(nb, goal)
            heapq.heappush(heap, (new_g + h, new_g, next(tie), nb))

    return None # Shouldn't happen for solvable instances

def print_board(state: State) -> None:
    """Pretty-print a 3x3 board."""
    for r in range(3):
        row = state[3 * r : 3 * r + 3]
        print(" ".join(str(x) if x != 0 else "." for x in row))
    print()

def parse_state(s: str) -> State:
    """
    Parse a state from a string like "1,2,3,4,0,6,7,5,8" or "123406758".
    """
    s = s.strip().replace(" ", "")
    if "," in s:
        nums = [int(x) for x in s.split(",")]
    else:
        if len(s) != 9 or not s.isdigit():
            raise ValueError("Provide 9 digits including 0 for the blank, or 9 comma-separated numbers.")
        nums = [int(ch) for ch in s]
    if len(nums) != 9 or set(nums) != set(range(9)):
        raise ValueError("State must contain exactly the numbers 0..8 once each.")
    return tuple(nums) # type_ignore[return-value]

def main(start_state_str: str = "1,2,3,4,0,6,7,5,8", goal_state_str: str = "1,2,3,4,5,6,7,8,0",
        heuristic_name: str = "manhattan", show_path: bool = True):
    start = parse_state(start_state_str)

```

```

goal = parse_state(goal_state_str)

# Reset goal positions cache if using a different goal
_goal_positions.cache_clear()

heur: Callable[[State, State], int] = manhattan_distance if heuristic_name == "manhattan" \
    else misplaced_tiles

result = a_star(start, heuristic=heur, goal=goal)
if result is None:
    print("This puzzle is unsolvable.")
    return

path: List[State] = result["path"] # type_ignore[assignment]
moves: int = result["moves"]      # type_ignore[assignment]
expanded: int = result["expanded_nodes"] # type_ignore[assignment]
seq: str = result.get("move_sequence", "") # type_ignore[assignment]

print(f"Solved in {moves} moves using {heuristic_name} (expanded {expanded} states).")
print(f"Move sequence: {seq}")
if show_path:
    print("\nPath:")
    for i, st in enumerate(path):
        print(f"Step {i}:")
        print_board(st)

main()

# iddfs
import heapq
from itertools import count
from typing import Callable, Dict, Iterable, List, Optional, Tuple
from functools import lru_cache

State = Tuple[int, ...] # 9-length tuple; 0 represents the blank
GOAL: State = (1, 2, 3, 4, 5, 6, 7, 8, 0)

# Directions for blank movement: (dr, dc, move_char)
DIRS = [(-1, 0, 'U'), (1, 0, 'D'), (0, -1, 'L'), (0, 1, 'R')]

def index_to_rc(i: int) -> Tuple[int, int]:
    return divmod(i, 3)

def rc_to_index(r: int, c: int) -> int:
    return r * 3 + c

```

```

def is_solvable(state: State) -> bool:
    """An 8-puzzle is solvable iff the inversion count (ignoring 0) is even."""
    arr = [x for x in state if x != 0]
    inv = 0
    for i in range(len(arr)):
        for j in range(i + 1, len(arr)):
            if arr[i] > arr[j]:
                inv += 1
    return inv % 2 == 0

```

```

def neighbors(state: State) -> Iterable[Tuple[State, str]]:
    """Yield (neighbor_state, move_char) for each legal slide."""
    i0 = state.index(0)
    r0, c0 = index_to_rc(i0)
    for dr, dc, mv in DIRS:
        nr, nc = r0 + dr, c0 + dc
        if 0 <= nr < 3 and 0 <= nc < 3:
            ni = rc_to_index(nr, nc)
            new_state = list(state)
            new_state[i0], new_state[ni] = new_state[ni], new_state[i0]
            yield tuple(new_state), mv

```

```

def reconstruct_path(parents: Dict[State, State], end: State) -> List[State]:
    path = [end]
    while end in parents:
        end = parents[end]
        path.append(end)
    path.reverse()
    return path

```

```

def path_to_moves(path: List[State]) -> str:
    """Convert a path of states into a string of moves 'UDLR'."""
    moves = []
    for a, b in zip(path, path[1:]):
        ia, ib = a.index(0), b.index(0)
        ra, ca = index_to_rc(ia)
        rb, cb = index_to_rc(ib)
        if rb == ra - 1 and cb == ca:
            moves.append('U')
        elif rb == ra + 1 and cb == ca:
            moves.append('D')
        elif rb == ra and cb == ca - 1:

```



```

        moves.append('L')
    elif rb == ra and cb == ca + 1:
        moves.append('R')
    else:
        moves.append('?')
    return ".join(moves)

```

```

def misplaced_tiles(state: State, goal: State = GOAL) -> int:
    """Heuristic: number of tiles not in their goal position (ignore blank)."""
    return sum(1 for i, v in enumerate(state) if v != 0 and v != goal[i])

```

```

@lru_cache(maxsize=None)
def _goal_positions(goal: State) -> Tuple[Tuple[int, int], ...]:
    """Cache goal positions as a tuple indexed by tile value (0..8)."""
    pos = [(0, 0)] * 9
    for i, v in enumerate(goal):
        r, c = index_to_rc(i)
        pos[v] = (r, c)
    return tuple(pos)

```

```

def manhattan_distance(state: State, goal: State = GOAL) -> int:
    """Heuristic: sum of Manhattan distances of each tile to its goal position."""
    pos = _goal_positions(goal)
    total = 0
    for i, v in enumerate(state):
        if v == 0:
            continue
        r, c = index_to_rc(i)
        gr, gc = pos[v]
        total += abs(r - gr) + abs(c - gc)
    return total

```

```

def a_star(
    start: State,
    heuristic: Callable[[State, State], int] = manhattan_distance,
    goal: State = GOAL,
) -> Optional[Dict[str, object]]:
    """
    A* search for the 8-puzzle.
    Returns a dict with keys: path, moves, expanded_nodes, or None if unsolvable.
    """
    if start == goal:
        return {"path": [start], "moves": 0, "expanded_nodes": 0}

```

```

if not is_solvable(start):
    return None

# Each heap item: (f, g, tie, state)
heap: List[Tuple[int, int, int, State]] = []
tie = count()
g0 = 0
h0 = heuristic(start, goal)
heapq.heappush(heap, (g0 + h0, g0, next(tie), start))

best_g: Dict[State, int] = {start: 0}
parents: Dict[State, State] = {}
expanded = 0

while heap:
    f, g, _, state = heapq.heappop(heap)
    expanded += 1

    if state == goal:
        path = reconstruct_path(parents, state)
        return {
            "path": path,
            "moves": len(path) - 1,
            "expanded_nodes": expanded,
            "move_sequence": path_to_moves(path),
        }

    for nb, _mv in neighbors(state):
        new_g = g + 1
        if nb not in best_g or new_g < best_g[nb]:
            best_g[nb] = new_g
            parents[nb] = state
            h = heuristic(nb, goal)
            heapq.heappush(heap, (new_g + h, new_g, next(tie), nb))

return None # Shouldn't happen for solvable instances

def print_board(state: State) -> None:
    """Pretty-print a 3x3 board."""
    for r in range(3):
        row = state[3 * r : 3 * r + 3]
        print(" ".join(str(x) if x != 0 else "." for x in row))
    print()

```

```

def parse_state(s: str) -> State:
    """
    Parse a state from a string like "1,2,3,4,0,6,7,5,8" or "123406758".
    """
    s = s.strip().replace(" ", "")
    if "," in s:
        nums = [int(x) for x in s.split(",")]
    else:
        if len(s) != 9 or not s.isdigit():
            raise ValueError("Provide 9 digits including 0 for the blank, or 9 comma-separated numbers.")
        nums = [int(ch) for ch in s]
    if len(nums) != 9 or set(nums) != set(range(9)):
        raise ValueError("State must contain exactly the numbers 0..8 once each.")
    return tuple(nums) # type_ignore[return-value]

def main(start_state_str: str = "1,2,3,4,0,6,7,5,8", goal_state_str: str = "1,2,3,4,5,6,7,8,0",
        heuristic_name: str = "manhattan", show_path: bool = True):
    start = parse_state(start_state_str)
    goal = parse_state(goal_state_str)

    # Reset goal positions cache if using a different goal
    _goal_positions.cache_clear()

    heur: Callable[[State, State], int] = manhattan_distance if heuristic_name == "manhattan" \
        else misplaced_tiles

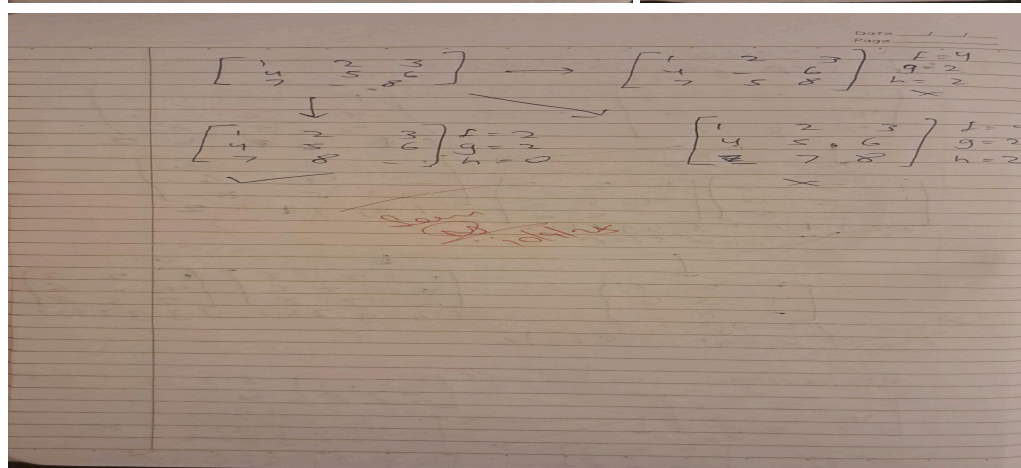
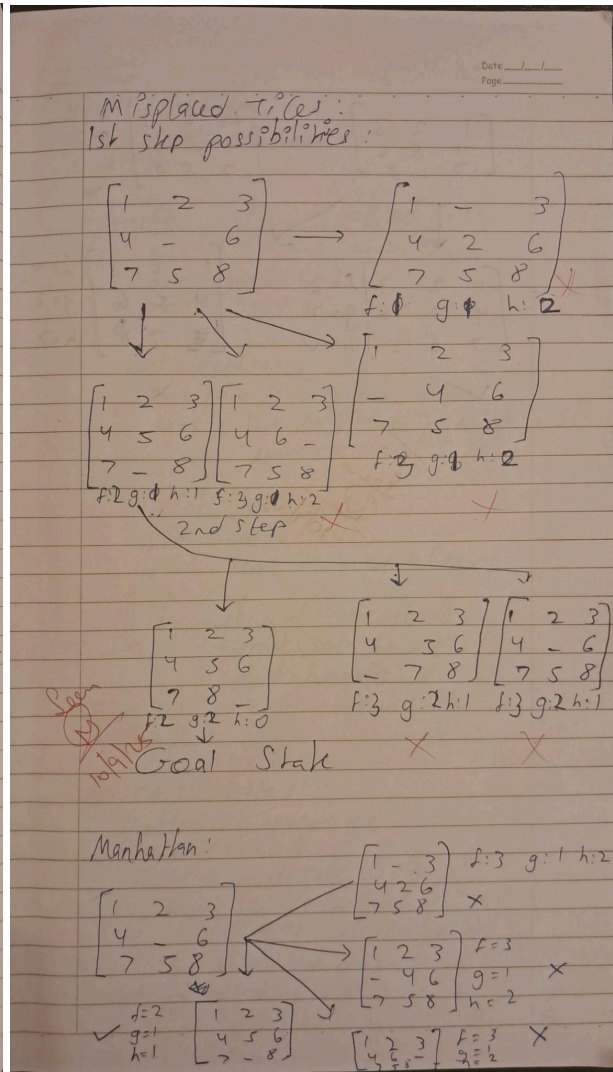
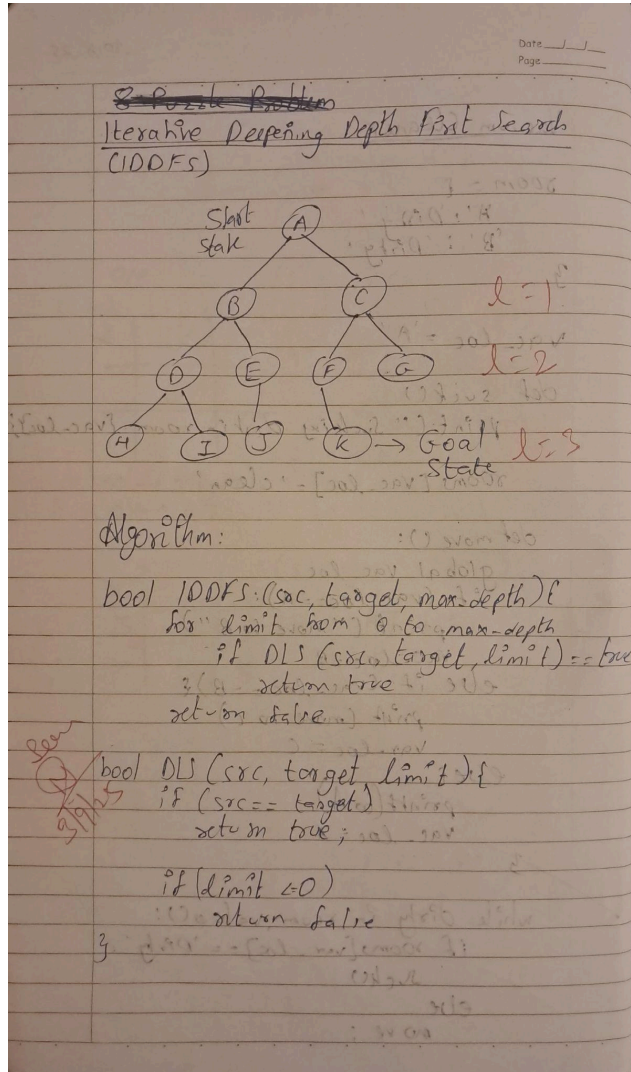
    result = a_star(start, heuristic=heur, goal=goal)
    if result is None:
        print("This puzzle is unsolvable.")
        return

    path: List[State] = result["path"] # type_ignore[assignment]
    moves: int = result["moves"] # type_ignore[assignment]
    expanded: int = result["expanded_nodes"] # type_ignore[assignment]
    seq: str = result.get("move_sequence", "") # type_ignore[assignment]

    print(f"Solved in {moves} moves using {heuristic_name} (expanded {expanded} states).")
    print(f"Move sequence: {seq}")
    if show_path:
        print("\nPath:")
        for i, st in enumerate(path):
            print(f"Step {i}:")
            print_board(st)

main()

```



Program 3

Implement A* search algorithm

Algorithm:

(You should provide the screen shot of your observation book of Algorithm/Logic/Solving of respective problem)

Code:

```
class Node:
    def __init__(self,data,level,fval):
        """ Initialize the node with the data, level of the node and the calculated fvalue """
        self.data = data
        self.level = level
        self.fval = fval

    def generate_child(self):
        """ Generate child nodes from the given node by moving the blank space
            either in the four directions {up,down,left,right} """
        x,y = self.find(self.data,'_')
        """ val_list contains position values for moving the blank space in either of
            the 4 directions [up,down,left,right] respectively. """
        val_list = [[x,y-1],[x,y+1],[x-1,y],[x+1,y]]
        children = []
        for i in val_list:
            child = self.shuffle(self.data,x,y,i[0],i[1])
            if child is not None:
                child_node = Node(child,self.level+1,0)
                children.append(child_node)
        return children

    def shuffle(self,puz,x1,y1,x2,y2):
        """ Move the blank space in the given direction and if the position value are out
            of limits the return None """
        if x2 >= 0 and x2 < len(self.data) and y2 >= 0 and y2 < len(self.data):
            temp_puz = []
            temp_puz = self.copy(puz)
            temp = temp_puz[x2][y2]
            temp_puz[x2][y2] = temp_puz[x1][y1]
            temp_puz[x1][y1] = temp
            return temp_puz
        else:
            return None

    def copy(self,root):
        """ Copy function to create a similar matrix of the given node"""
        temp = []
```

```

    for i in root:
        t = []
        for j in i:
            t.append(j)
        temp.append(t)
    return temp

def find(self,puz,x):
    """ Specifically used to find the position of the blank space """
    for i in range(0,len(self.data)):
        for j in range(0,len(self.data)):
            if puz[i][j] == x:
                return i,j

class Puzzle:
    def __init__(self,size):
        """ Initialize the puzzle size by the specified size,open and closed lists to empty """
        self.n = size
        self.open = []
        self.closed = []

    def accept(self):
        """ Accepts the puzzle from the user """
        puz = []
        for i in range(0,self.n):
            temp = input().split(" ")
            puz.append(temp)
        return puz

    def f(self,start,goal):
        """ Heuristic Function to calculate hueristic value  $f(x) = h(x) + g(x)$  """
        return self.h(start.data,goal)+start.level

    def h(self,start,goal):
        """ Calculates the different between the given puzzles """
        temp = 0
        for i in range(0,self.n):
            for j in range(0,self.n):
                if start[i][j] != goal[i][j] and start[i][j] != '_':
                    temp += 1
        return temp

    def process(self):
        """ Accept Start and Goal Puzzle state"""
        print("Enter the start state matrix \n")

```

```

start = self.accept()
print("Enter the goal state matrix \n")
goal = self.accept()

start = Node(start,0,0)
start.fval = self.f(start,goal)
""" Put the start node in the open list"""
self.open.append(start)
print("\n\n")
while True:
    cur = self.open[0]
    print("")
    print(" | ")
    print(" | ")
    print(" \\\\/ \n")
    for i in cur.data:
        for j in i:
            print(j,end=" ")
        print("")
    """ If the difference between current and goal node is 0 we have reached the goal node"""
    if(self.h(cur.data,goal) == 0):
        break
    for i in cur.generate_child():
        i.fval = self.f(i,goal)
        self.open.append(i)
    self.closed.append(cur)
    del self.open[0]

    """ sort the opne list based on f value """
    self.open.sort(key = lambda x:x.fval,reverse=False)

puz = Puzzle(3)
puz.process()

```

8 puzzle problem (A*)

class Puzzle:

def __init__(self, size):

self.n = size

self.open = []

self.close = []

def accept(self):

puz = []

for i in range(0, self.n):

temp = input().split(" ")

puz.append(temp)

return puz

def f(self, start, goal):

return self.h(start.data, goal) + start.level

def h(self, start, goal):

temp = 0

for i in range(0, self.n):

temp += 1

return temp

def process(self):

start = self.accept()

goal = self.accept()

start = Node(start, 0, 0)

start.fval = self.f(start, goal)

puz = Puzzle(3)

puz.process()

O/P:

Enter the start state matrix

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & - & 6 \\ 7 & 5 & 8 \end{bmatrix}$$

Enter the goal state matrix

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & - \end{bmatrix}$$

O/P:

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & - & 6 \\ 7 & 5 & 8 \end{bmatrix} \longrightarrow \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & - & 8 \end{bmatrix}$$
$$\downarrow$$
$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & - \end{bmatrix}$$

Program 4

Implement Hill Climbing search algorithm to solve N-Queens problem

Algorithm:

(You should provide the screen shot of your observation book of Algorithm/Logic/Solving of respective problem)

Code:

```
def print_board(state):
    """Prints the 4x4 board representation with 'Q' and '.'."""
    n = len(state)
    for row in range(n):
        for col in range(n):
            if state[col] == row:
                print("Q", end=" ")
            else:
                print(".", end=" ")
        print()
    print()

def calculate_cost(state):
    """Returns number of attacking pairs of queens."""
    cost = 0
    n = len(state)
    for i in range(n):
        for j in range(i + 1, n):
            # same row
            if state[i] == state[j]:
                cost += 1
            # same diagonal
            elif abs(state[i] - state[j]) == abs(i - j):
                cost += 1
    return cost

def get_neighbors(state):
    """Generates all neighbors by swapping two queen positions."""
    neighbors = []
    n = len(state)
    for i in range(n):
        for j in range(i + 1, n):
            neighbor = state.copy()
            neighbor[i], neighbor[j] = neighbor[j], neighbor[i]
            neighbors.append((neighbor, (i, j)))
    return neighbors
```

```

def hill_climbing(state):
    print("\nInitial State:", state)
    print_board(state)
    current_cost = calculate_cost(state)
    step = 1

    while True:
        print(f'Step {step}: Current cost = {current_cost}')
        neighbors = get_neighbors(state)
        neighbor_costs = []

        # Calculate cost for all neighbors
        for neighbor, swapped in neighbors:
            cost = calculate_cost(neighbor)
            neighbor_costs.append((cost, neighbor, swapped))

        # Sort by cost and then by smallest column pair
        neighbor_costs.sort(key=lambda x: (x[0], x[2][0], x[2][1]))

        # Display neighbor costs
        print("Neighbor states and their costs:")
        for cost, neighbor, swapped in neighbor_costs:
            print(f'Swap x {swapped[0]} & x {swapped[1]} => {neighbor}, Cost = {cost}')

        best_cost, best_state, swap = neighbor_costs[0]
        print("\nBest Neighbor after swap", swap, "is", best_state, "with cost =", best_cost)
        print_board(best_state)

        if best_cost >= current_cost: # No improvement (local minimum)
            print("No better neighbor found. Hill Climbing terminated.")
            print("Final state:", state)
            print_board(state)
            break
        else:
            state = best_state
            current_cost = best_cost

        if current_cost == 0:
            print("Goal state reached!")
            print_board(state)
            break

        step += 1

# ----- MAIN -----

```

```

if __name__ == "__main__":
    print("Hill Climbing for 4-Queens Problem")
    print("Enter the row positions of 4 queens (each between 0 and 3):")
    state = list(map(int, input("Example (1 2 0 3): ").split()))
    hill_climbing(state)

```

Lab-5

1) Hill Climbing
2) Simulated Annealing

1) Function HILL-CLIMBING(s):

```

print("initial state")
print-board(s)
print "cost = ", cost(s)

loop forever:
    neigh = get-neighbours(s)
    next = the neighbour in neigh with
            lowest cost.

    print "next state"
    print-board(next)
    print "cost = ", cost(next)

    if cost(next) >= cost(s):
        print("soln found")
        print-board(s)

```

Code:

```

def cost(state):
    attacking-pairs = 0
    n = len(state)
    for i in range(n):
        for j in range(i+1, n):
            if state[i] == state[j]:
                attacking-pairs += 1
    return attacking-pairs

```

```

def get-neighbours(state):
    neighbours = []
    n = len(state)
    for i in range(n):
        for j in range(i+1, n):
            neighbour = list(state)
            neighbour[i], neighbour[j] = n[j], n[i]
            neighbours.append(tuple(neighbour))
    return neighbours

def hill-climb(state):
    current = state
    print-board(current)

    while True:
        neigh = get-neighbours(state)
        next-state = min(neighbours, key=lambda x: cost(x))
        print-board(next-state)

        if cost(next-state) >= cost(current):
            print-board(current)
            return current
        current = next-state

```

O/P:

. . . Q	. . . Q
. Q . .	Q . . .
. . Q .	. . Q .
Q . . .	. Q . .
cost = 2	cost = 1

Program 5

Simulated Annealing to Solve 8-Queens problem

Algorithm:

(You should provide the screen shot of your observation book of Algorithm/Logic/Solving of respective problem)

Code:

```
import random
import math

def print_board(state):
    """Prints the 4x4 board representation with 'Q' and '.'"""
    n = len(state)
    print("Board State:")
    for row in range(n):
        line = ""
        for col in range(n):
            if state[col] == row:
                line += "Q "
            else:
                line += ". "
        print(line)
    print()

def calculate_cost(state):
    """Returns the number of attacking pairs of queens (heuristic cost)."""
    cost = 0
    n = len(state)
    for i in range(n):
        for j in range(i + 1, n):
            # Check for queens in the same row or on the same diagonal
            if state[i] == state[j] or abs(state[i] - state[j]) == abs(i - j):
                cost += 1
    return cost

# ----- HILL CLIMBING ALGORITHM -----

def get_all_neighbors(state):
    """Generates all possible neighbors by swapping two queen positions."""
    neighbors = []
    n = len(state)
    for i in range(n):
        for j in range(i + 1, n):
            neighbor = state.copy()
            neighbor[i], neighbor[j] = neighbor[j], neighbor[i]
```

```

        neighbors.append((neighbor, (i, j)))
    return neighbors

def hill_climbing(state):
    """
    Solves the 4-Queens problem using steepest-ascent Hill Climbing.
    It evaluates all neighbors and picks the best one.
    """
    print("\n--- Starting Hill Climbing ---")
    current_state = state.copy()
    current_cost = calculate_cost(current_state)
    step = 1

    print("Initial State:", current_state, "with Cost:", current_cost)
    print_board(current_state)

    while True:
        print(f"--- Step {step} ---")
        neighbors = get_all_neighbors(current_state)
        neighbor_costs = []

        # Calculate cost for all neighbors
        for neighbor, swapped in neighbors:
            cost = calculate_cost(neighbor)
            neighbor_costs.append((cost, neighbor, swapped))

        # Sort by best cost, then by the smallest column pair as a tie-breaker
        neighbor_costs.sort(key=lambda x: (x[0], x[2][0], x[2][1]))

        best_cost, best_neighbor, best_swap = neighbor_costs[0]

        print(f"Current State: {current_state}, Cost: {current_cost}")
        print(f"Best neighbor is {best_neighbor} with cost {best_cost} (Swap columns {best_swap[0]}
& {best_swap[1]})")

        # If the best neighbor is not better than the current state, we are stuck
        if best_cost >= current_cost:
            print("\nNo better neighbor found. Reached a local minimum or plateau.")
            print("Final State:", current_state)
            print_board(current_state)
            print("Final Cost:", current_cost)
            break

        # Move to the best neighbor
        current_state = best_neighbor
        current_cost = best_cost

```

```

print_board(current_state)

# If goal is reached
if current_cost == 0:
    print("\nGoal state reached! ðŸŽ‰")
    print("Final State:", current_state)
    print_board(current_state)
    print("Final Cost:", current_cost)
    break

step += 1

# ----- SIMULATED ANNEALING ALGORITHM -----

def get_random_neighbor(state):
    """Generates a single random neighbor by swapping two queen positions."""
    n = len(state)
    neighbor = state.copy()
    # Pick two distinct random columns to swap
    i, j = random.sample(range(n), 2)
    neighbor[i], neighbor[j] = neighbor[j], neighbor[i]
    return neighbor

def simulated_annealing(state, initial_temp=100.0, cooling_rate=0.95, max_steps=1000):
    """
    Solves the 4-Queens problem using Simulated Annealing.
    It explores the search space by sometimes accepting worse moves.
    """
    print("\n--- Starting Simulated Annealing ---")
    current_state = state.copy()
    current_cost = calculate_cost(current_state)
    temp = initial_temp

    print("Initial State:", current_state, "with Cost:", current_cost)
    print_board(current_state)

    for step in range(max_steps):
        if temp < 0.01:
            print("\nTemperature cooled down. Algorithm terminated.")
            break

        # Check for goal state
        if current_cost == 0:
            print(f"\nGoal state reached in {step+1} steps! ðŸŽ‰")
            break

```

```

# Generate a random neighbor
neighbor_state = get_random_neighbor(current_state)
neighbor_cost = calculate_cost(neighbor_state)

# Calculate the change in cost (energy)
delta_cost = neighbor_cost - current_cost

# Decide whether to move to the neighbor
# Always move if the neighbor is better
if delta_cost < 0:
    print(f'Step {step+1}: Good move. Moved to {neighbor_state} (Cost: {neighbor_cost})')
    current_state = neighbor_state
    current_cost = neighbor_cost
# If neighbor is worse, maybe move based on probability
else:
    acceptance_prob = math.exp(-delta_cost / temp)
    if random.random() < acceptance_prob:
        print(f'Step {step+1}: Bad move accepted! Moved to {neighbor_state} (Cost:
{neighbor_cost}) with prob {acceptance_prob:.2f}')
        current_state = neighbor_state
        current_cost = neighbor_cost
    else:
        print(f'Step {step+1}: Bad move rejected. Staying at {current_state} (Cost:
{current_cost})')

# Cool the temperature
temp *= cooling_rate

print("\nFinal State:", current_state)
print_board(current_state)
print("Final Cost:", current_cost)

# ----- MAIN EXECUTION -----
if __name__ == "__main__":
    print("--- 4-Queens Problem Solver ---")
    print("The state is a list of 4 numbers representing the row of the queen in each column.")
    print("For example, '1 3 0 2' means:")
    print(" - Queen in column 0 is in row 1")
    print(" - Queen in column 1 is in row 3")
    print(" - Queen in column 2 is in row 0")
    print(" - Queen in column 3 is in row 2\n")

    try:
        initial_state = list(map(int, input("Enter the initial state (e.g., 2 0 3 1): ").split()))

```

```

if len(initial_state) != 4 or not all(0 <= x <= 3 for x in initial_state):
    print("Invalid input. Please enter 4 numbers, each between 0 and 3.")
else:
    simulated_annealing(initial_state)

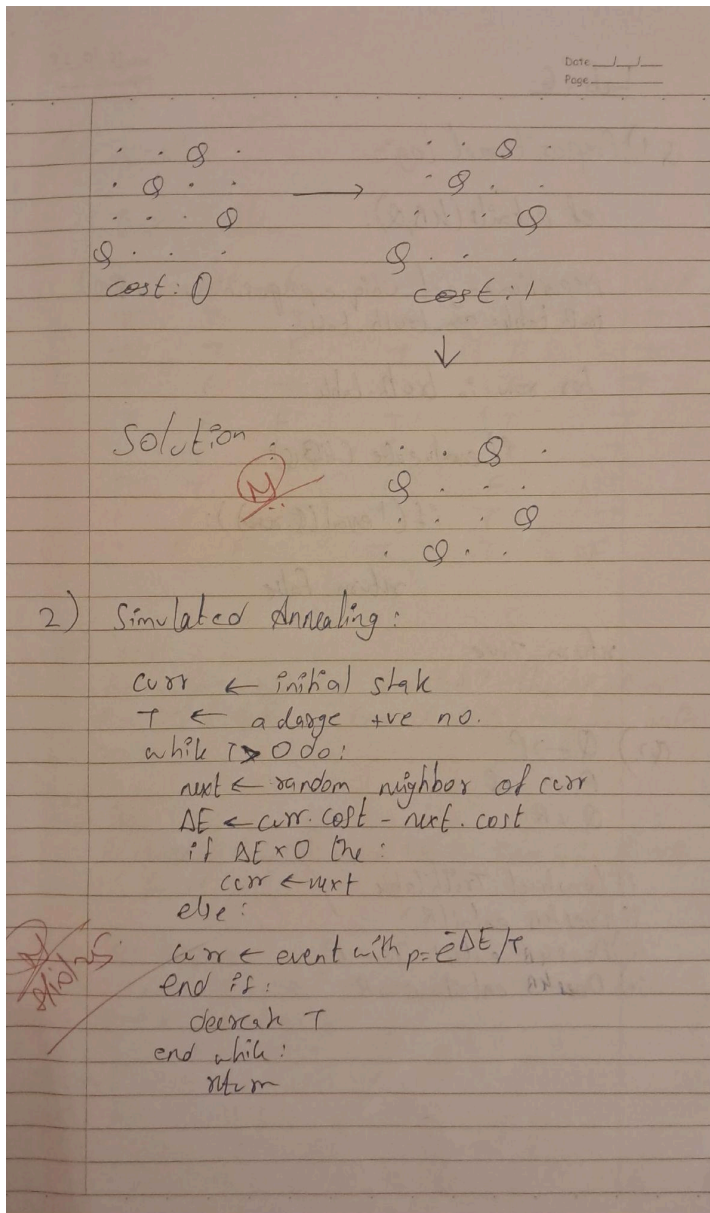
```

except ValueError:

```

print("Invalid input format. Please enter numbers separated by spaces.")

```



Program 6

Create a knowledge base using propositional logic and show that the given query entails the knowledge base or not.

Algorithm:

(You should provide the screen shot of your observation book of Algorithm/Logic/Solving of respective problem)

Code:

```
import itertools
import pandas as pd

variables = ['P', 'Q', 'R']

def q_implies_p(p, q, r):
    return not q or p

def p_implies_not_q(p, q, r):
    return not p or not q

def q_or_r(p, q, r):
    return q or r

sentences = {
    'Q implies P': q_implies_p,
    'P implies not(Q)': p_implies_not_q,
    'Q or R': q_or_r
}

truth_values = [True, False]
combinations = list(itertools.product(truth_values, repeat=len(variables)))

truth_table_data = []
for combination in combinations:
    p, q, r = combination
    row = {'P': p, 'Q': q, 'R': r}
    for sentence_name, sentence_func in sentences.items():
        row[sentence_name] = sentence_func(p, q, r)
    truth_table_data.append(row)

truth_table_df = pd.DataFrame(truth_table_data)

sentence_columns = list(sentences.keys())
truth_table_df['KB is True'] = truth_table_df[sentence_columns].all(axis=1)
```

```
models = truth_table_df[truth_table_df['KB is True']]
```

```
kb_entails_r = models['R'].all()
```

```
kb_entails_r_implies_p = (models['R'].apply(lambda x: not x) | models['P']).all()
```

```
kb_entails_q_implies_r = (models['Q'].apply(lambda x: not x) | models['R']).all()
```

```
print("Truth Table:")
```

```
print(truth_table_df)
```

```
print("\nModels where KB is True:")
```

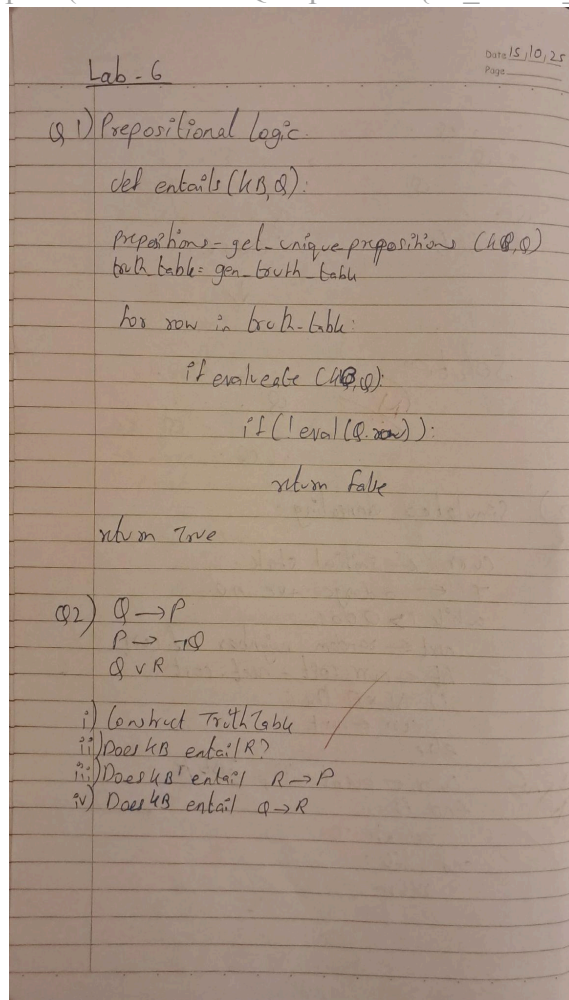
```
print(models)
```

```
print("\nEntailment Results:")
```

```
print(f'KB entails R: {kb_entails_r}')
```

```
print(f'KB entails R implies P: {kb_entails_r_implies_p}')
```

```
print(f'KB entails Q implies R: {kb_entails_q_implies_r}')
```



Program 7

Implement unification in first order logic

Algorithm:

(You should provide the screen shot of your observation book of Algorithm/Logic/Solving of respective problem)

Code:

```
import re

# Utility: parse the expression into function/operator and arguments
def parse(expr):
    expr = expr.strip()
    if '(' not in expr:
        return expr, []
    func = expr[:expr.index('(')].strip()
    args = expr[expr.index('(')+1:-1]
    args = [a.strip() for a in split_args(args)]
    return func, args

# Split arguments correctly (handles nested brackets)
def split_args(args_str):
    args, level, start = [], 0, 0
    for i, ch in enumerate(args_str):
        if ch == ',' and level == 0:
            args.append(args_str[start:i].strip())
            start = i + 1
        elif ch == '(':
            level += 1
        elif ch == ')':
            level -= 1
    args.append(args_str[start:].strip())
    return args

# Apply substitution to an expression
def substitute(expr, subs):
    for var, val in subs.items():
        expr = re.sub(rf'\b{var}\b', val, expr)
    return expr

# Check if variable occurs inside term (Occurs check)
def occurs_check(var, term):
    if var == term:
        return True
    if '(' not in term:
        return False
    _, args = parse(term)
```

```

    return any(occurs_check(var, arg) for arg in args)

# Unification algorithm
def unify(e1, e2, subs=None):
    if subs is None:
        subs = {}

    e1 = substitute(e1, subs)
    e2 = substitute(e2, subs)

    if e1 == e2:
        return subs

    f1, args1 = parse(e1)
    f2, args2 = parse(e2)

    # Case 1: Both are compound terms
    if args1 and args2:
        if f1 != f2 or len(args1) != len(args2):
            print(f'Function symbols or arity mismatch: {f1} vs {f2}')
            return None
        for a1, a2 in zip(args1, args2):
            subs = unify(a1, a2, subs)
            if subs is None:
                return None
        return subs

    # Case 2: Variable binding
    elif e1.islower() and e1.isalpha(): # e1 is variable
        if occurs_check(e1, e2):
            print(f'Occurs check failed: {e1} occurs in {e2}')
            return None
        subs[e1] = e2
        return subs
    elif e2.islower() and e2.isalpha(): # e2 is variable
        if occurs_check(e2, e1):
            print(f'Occurs check failed: {e2} occurs in {e1}')
            return None
        subs[e2] = e1
        return subs

    # Otherwise mismatch
    else:
        print(f'Cannot unify {e1} with {e2}')
        return None

# --- MAIN PROGRAM ---

```

```
print("=== Unification Algorithm ===")
expr1 = input("Enter first expression: ").strip()
expr2 = input("Enter second expression: ").strip()
```

```
result = unify(expr1, expr2)
```

```
if result:
```

```
    print("\n Unification Successful!")
```

```
    print("Substitutions:")
```

```
    for k, v in result.items():
```

```
        print(f' {k} / {v}')
```

```
else:
```

```
    print("\n Unification Failed.")
```

Date: __/__/__
Page: _____

Q. 10

i) Truth Table

P	Q	R	$Q \rightarrow P$	$P \rightarrow \neg Q$	$Q \vee R$
T	T	T	T	F	T
T	T	F	T	F	T
T	F	T	T	T	T
T	F	F	T	T	T
F	T	T	F	T	T
F	T	F	F	F	T
F	F	T	T	T	T
F	F	F	T	T	F

ii) Yes.
Because considering row 3 & 7, the KB $Q \rightarrow P$, $P \rightarrow \neg Q$ & $Q \vee R$ is true & R is true for both rows.

iii) No
Because Row 7 model KB is True but $R \rightarrow Q$ is false.

iv) Yes because
Because row 3 & 7 all models are true and $Q \rightarrow R$ is also true.

Proved

Program 8

Create a knowledge base consisting of first order logic statements and prove the given query using forward reasoning.

Algorithm:

(You should provide the screen shot of your observation book of Algorithm/Logic/Solving of respective problem)

Code:

```
import re

def isVariable(x):
    return len(x) == 1 and x.islower() and x.isalpha()

def getAttributes(string):
    expr = r'\([^)]+\)'
    matches = re.findall(expr, string)
    return matches

def getPredicates(string):
    expr = r'([a-z~]+\)(\[^\&]+\)'
    return re.findall(expr, string)

class Fact:
    def __init__(self, expression):
        self.expression = expression
        predicate, params = self.splitExpression(expression)
        self.predicate = predicate
        self.params = params
        self.result = any(self.getConstants())

    def splitExpression(self, expression):
        predicate = getPredicates(expression)[0]
        params = getAttributes(expression)[0].strip('(')').split(',')
        return [predicate, params]

    def getResult(self):
        return self.result

    def getConstants(self):
        return [None if isVariable(c) else c for c in self.params]

    def getVariables(self):
        return [v if isVariable(v) else None for v in self.params]

    def substitute(self, constants):
        constants_copy = constants.copy()
```

```

    expr = f'{self.predicate}({','.join([constants_copy.pop(0) if isVariable(p) else p for p in
self.params])})'
    return Fact(expr)

```

```

class Implication:

```

```

    def __init__(self, expression):
        self.expression = expression
        l = expression.split('=>')
        self.lhs = [Fact(f) for f in l[0].split('&')]
        self.rhs = Fact(l[1])

```

```

    def evaluate(self, facts):
        constants = {}
        new_lhs = []
        for fact in facts:
            for val in self.lhs:
                if val.predicate == fact.predicate:
                    for i, v in enumerate(val.getVariables()):
                        if v:
                            constants[v] = fact.getConstants()[i]
                    new_lhs.append(fact)

```

```

        predicate = getPredicates(self.rhs.expression)[0]
        attributes = str(getAttributes(self.rhs.expression)[0])

```

```

        for key in constants:
            if constants[key]:
                attributes = attributes.replace(key, constants[key])

```

```

        expr = f'{predicate} {attributes}'
        return Fact(expr) if len(new_lhs) and all([f.getResult() for f in new_lhs]) else None

```

```

class KB:

```

```

    def __init__(self):
        self.facts = set()
        self.implications = set()

```

```

    def tell(self, e):
        if '=>' in e:
            self.implications.add(Implication(e))
        else:
            self.facts.add(Fact(e))
        for i in self.implications:
            res = i.evaluate(self.facts)
            if res:
                self.facts.add(res)

```

```

def ask(self, e):
    facts = set([f.expression for f in self.facts])
    print(f'\nQuerying {e}:')
    i = 1
    found = False
    for f in facts:
        if Fact(f).predicate == Fact(e).predicate:
            print(f'\t{i}. {f}')
            i += 1
            found = True
    if not found:
        print("\tNo matching facts found.")

def display(self):
    print("\nAll facts:")
    for i, f in enumerate(set([f.expression for f in self.facts])):
        print(f'\t{i+1}. {f}')

def main():
    kb = KB()
    print("Enter the number of FOL expressions present in KB:")
    n = int(input())
    print("Enter the expressions:")
    for i in range(n):
        fact = input().strip()
        kb.tell(fact)

    print("Enter the query:")
    query = input().strip()
    kb.ask(query)
    kb.display()

if __name__ == "__main__":
    main()
import re

def isVariable(x):
    return len(x) == 1 and x.islower() and x.isalpha()

def getAttributes(string):
    expr = r"([^\s]+)"
    matches = re.findall(expr, string)
    return matches

def getPredicates(string):
    expr = r"([a-z~]+)([^\s]+)"
    return re.findall(expr, string)

```



```

class Fact:
    def __init__(self, expression):
        self.expression = expression
        predicate, params = self.splitExpression(expression)
        self.predicate = predicate
        self.params = params
        self.result = any(self.getConstants())

    def splitExpression(self, expression):
        predicate = getPredicates(expression)[0]
        params = getAttributes(expression)[0].strip('(').split(',')
        return [predicate, params]

    def getResult(self):
        return self.result

    def getConstants(self):
        return [None if isVariable(c) else c for c in self.params]

    def getVariables(self):
        return [v if isVariable(v) else None for v in self.params]

    def substitute(self, constants):
        constants_copy = constants.copy()
        expr = f'{self.predicate}({','.join([constants_copy.pop(0) if isVariable(p) else p for p in self.params])})'
        return Fact(expr)

class Implication:
    def __init__(self, expression):
        self.expression = expression
        l = expression.split('=>')
        self.lhs = [Fact(f) for f in l[0].split('&')]
        self.rhs = Fact(l[1])

    def evaluate(self, facts):
        constants = {}
        new_lhs = []
        for fact in facts:
            for val in self.lhs:
                if val.predicate == fact.predicate:
                    for i, v in enumerate(val.getVariables()):
                        if v:
                            constants[v] = fact.getConstants()[i]
                    new_lhs.append(fact)

        predicate = getPredicates(self.rhs.expression)[0]

```

```

attributes = str(getAttributes(self.rhs.expression)[0])

for key in constants:
    if constants[key]:
        attributes = attributes.replace(key, constants[key])

expr = f'{predicate} {attributes}'
return Fact(expr) if len(new_lhs) and all([f.getResult() for f in new_lhs]) else None

class KB:
    def __init__(self):
        self.facts = set()
        self.implications = set()

    def tell(self, e):
        if '=>' in e:
            self.implications.add(Implication(e))
        else:
            self.facts.add(Fact(e))
        for i in self.implications:
            res = i.evaluate(self.facts)
            if res:
                self.facts.add(res)

    def ask(self, e):
        facts = set([f.expression for f in self.facts])
        print(f'\nQuerying {e}:')
        i = 1
        found = False
        for f in facts:
            if Fact(f).predicate == Fact(e).predicate:
                print(f'\t{i}. {f}')
                i += 1
                found = True
        if not found:
            print("\tNo matching facts found.")

    def display(self):
        print("\nAll facts:")
        for i, f in enumerate(set([f.expression for f in self.facts])):
            print(f'\t{i+1}. {f}')

def main():
    kb = KB()
    print("Enter the number of FOL expressions present in KB:")
    n = int(input())
    print("Enter the expressions:")

```

```

for i in range(n):
    fact = input().strip()
    kb.tell(fact)

print("Enter the query:")
query = input().strip()
kb.ask(query)
kb.display()

if __name__ == "__main__":
    main()

```

Date: / /
Page:

2)

A	B	C	A ∨ C	B ∨ C	KB	Q
F	F	F	F	T	F	F
F	F	T	T	F	F	F
F	T	F	F	T	F	T
F	T	T	T	T	T	T
T	F	F	T	T	T	T
T	F	T	T	F	F	T
T	T	F	T	T	T	T
T	T	T	T	T	T	T

entailment

KB ⊨ Q: True

~~Q~~ ~~KB~~ ~~entailment~~

Date: 29/10/25
Page:

Lab-7: Unification

① $P(f(x), g(y), y)$
 $P(f(g(z)), g(f(a)), f(a))$
 find $\theta(MGUV)$

② $Q(x, f(x))$
 $Q(f(y), y)$

③ $H(x, g(x))$
 $H(g(y), g(g(z)))$

answers:

① $f(x) = f(g(z))$
 $x = g(z)$
 $g(y) = g(f(a))$
 $y = f(a)$
 $\therefore y = f(a)$ holds

2. $x = f(y)$
 No. of arguments are same
 $y = f(x)$
 $\therefore$ doesn't hold.

3. $x = g(y)$
 $g(x) = g(g(z))$

$$f(g(y)) = f(g(z))$$

$$g(y) = g(z) \quad g(z) = x$$

$\therefore$ Holds

$$y = z$$

$$g(y) = x$$

$$x = f(y)$$

$$y = f(x)$$

$$f(x) = y$$

$$x = f(f(x))$$

$$y = f(f(y))$$

~~Proceed~~

~~21/10/25~~

3. $x = g(y)$
 $g(x) = g(g(z))$

$$f(g(y)) = f(g(z))$$

$$g(y) = g(z) \quad g(z) = x$$

$\therefore$ Holds

$$y = z$$

$$g(y) = x$$

$$x = f(y)$$

$$y = f(x)$$

$$f(x) = y$$

$$x = f(f(x))$$

$$y = f(f(y))$$

~~Proceed~~

~~21/10/25~~

Program 9

Create a knowledge base consisting of first order logic statements and prove the given query using Resolution

Code:

```
def parse_clause(clause_str):
    return set(clause_str.split('v'))

def get_complement(literal):
    return literal[1:] if literal.startswith('~') else '~' + literal

def resolve(ci, cj):
    resolvents = set()
    for literal in ci:
        complement = get_complement(literal)
        if complement in cj:
            new_clause = (ci - {literal}) | (cj - {complement})
            resolvents.add(frozenset(new_clause))
    return resolvents

def resolution(kb_clauses, query):
    negated_query = get_complement(query)
    kb = [parse_clause(clause) for clause in kb_clauses] + [parse_clause(negated_query)]

    print("\n-----")
    print("KnowledgeBase - Resolution")
    print("-----")
    print(f"\nKnowledge Base Clauses: {kb_clauses}")
    print(f"Query: {query}")
    print(f"Negated Query Added: {negated_query}")
    print("\nResolution Steps:\n")

    new = set()

    while True:
        pairs = [(kb[i], kb[j]) for i in range(len(kb)) for j in range(i + 1, len(kb))]
        for (ci, cj) in pairs:
            resolvents = resolve(ci, cj)
            for resolvent in resolvents:
                print(f'Resolving {set(ci)} and {set(cj)} => {set(resolvent)}')
                if not resolvent:
                    print("\n Knowledge Base entails the query (empty clause derived).")
                    return True
```

```

        new.add(resolvent)

    if new.issubset(set(map(frozenset, kb))):
        print("\n Knowledge Base does NOT entail the query (no empty clause derived).")
        return False

    for clause in new:
        if clause not in kb:
            kb.append(clause)

print("KnowledgeBase - Resolution")
print("-----")
print("Enter clauses for the Knowledge Base.")
print("Use 'v' for OR between literals (e.g., '~qv~pvr'), and separate each clause with a space.\n")

kb_input = input("Enter clauses: ").split()
query_input = input("Enter the query: ")

resolution(kb_input, query_input)

```


Forward Chaining

for FOL-FC $\text{ask}(\text{KB}, \alpha)$ return a substitution or false.

inputs: KB a set of first order definite clauses. α , query, atomic sen.

local variable: the new sentence inferred on each iteration

repeat until new is empty.

new $\leftarrow \{\}$

for each rule in KB do:

($p_1 \dots p_n \rightarrow q$) $\leftarrow$

standardize variables (rule)

for each α such that match

($\alpha, p_1, \dots, p_n, p^*$):

subject ($\alpha, p_1, \dots, p_n$)

for some $p_1, \dots, p_n$ in KB

$q \leftarrow \text{subject}(\alpha, q)$

if q does not unify with

some sentence already

in KB or new then:

$\phi \leftarrow \text{unify}(q, \alpha)$

if ϕ is not false then return

ϕ and new to KB:

return false

Ex. John Likes Peanuts

$\neg \text{like}(\text{John}, \text{peanuts})$

$\neg \text{food}(x) \vee \text{likes}(x, y)$

$\text{food}(\text{peanuts})$

$\text{likes}(y, z) \vee \text{likes}(y, z)$

$\text{likes}(\text{John}, \text{peanuts}) \vee \text{likes}(y, z)$

$\text{likes}(\text{Anil}, \text{peanuts})$

$\text{likes}(\text{Anil})$

$\text{likes}(q) \vee \neg$

$\neg \text{likes}(\text{Anil})$

$\text{likes}(\text{Anil})$

$\{\}$

or.

0		x
x		
x	0	0

0		x
x	x	
x	0	0

0		x
x	x	x
x	0	0

0		x
x	x	x
x	0	0

Program 10

Implement Alpha-Beta Pruning.

Algorithm:

(You should provide the screen shot of your observation book of Algorithm/Logic/Solving of respective problem)

Code:

```
import math
import random

# Use an external "real" board only for the main game loop; recursive functions use state parameters.
board = [" " for _ in range(9)] # 3x3 board

def print_board(state):
    print("\n")
    for i in range(3):
        print(" " + " | ".join(state[i*3:(i+1)*3]))
        if i < 2:
            print("---+---+---")
    print("\n")

def is_winner(state, player):
    win_combinations = [
        [0, 1, 2], [3, 4, 5], [6, 7, 8],
        [0, 3, 6], [1, 4, 7], [2, 5, 8],
        [0, 4, 8], [2, 4, 6]
    ]
    return any(all(state[i] == player for i in combo) for combo in win_combinations)

def is_full(state):
    return " " not in state

def actions(state):
    return [i for i in range(9) if state[i] == " "]

def result(state, action, player):
    new_state = state.copy()
    new_state[action] = player
    return new_state

def utility(state):
    if is_winner(state, "O"):
        return +1
    elif is_winner(state, "X"):
        return -1
    else:
```



```

    return 0

def terminal_test(state):
    return is_winner(state, "X") or is_winner(state, "O") or is_full(state)

# --- Alpha-Beta Functions ---
def max_value(state, alpha, beta):
    if terminal_test(state):
        return utility(state)
    v = -math.inf
    for a in actions(state):
        v = max(v, min_value(result(state, a, "O"), alpha, beta))
        if v >= beta:
            return v
    alpha = max(alpha, v)
    return v

def min_value(state, alpha, beta):
    if terminal_test(state):
        return utility(state)
    v = math.inf
    for a in actions(state):
        v = min(v, max_value(result(state, a, "X"), alpha, beta))
        if v <= alpha:
            return v
    beta = min(beta, v)
    return v

def alpha_beta_search(state):
    best_score = -math.inf
    best_action = None
    if not actions(state):
        return None
    for a in actions(state):
        value = min_value(result(state, a, "O"), -math.inf, math.inf)
        if value > best_score:
            best_score = value
            best_action = a
    # Fallback: if something goes wrong, return a random legal move
    if best_action is None:
        legal = actions(state)
        return random.choice(legal) if legal else None
    return best_action

# --- Game Loop ---
def human_move():
    while True:

```

```

try:
    move = int(input("Enter your move (1-9): ")) - 1
except ValueError:
    print("Please enter a number 1-9.")
    continue
if move < 0 or move > 8:
    print("Move out of range. Choose 1-9.")
    continue
if board[move] != " ":
    print("Cell already taken. Try another.")
    continue
return move

def choose_first():
    while True:
        ans = input("Who goes first? (me/ai) [me]: ").strip().lower()
        if ans == "" or ans.startswith("m"):
            return "me"
        if ans.startswith("a"):
            return "ai"
        print("Type 'me' or 'ai' (or press Enter for me).")

def main():
    global board
    board = [" " for _ in range(9)]
    print("Welcome to Tic-Tac-Toe! You are X, AI is O.")
    first = choose_first()
    print_board(board)

    while True:
        if first == "me":
            # Human turn
            move = human_move()
            board[move] = "X"
            print_board(board)
            if is_winner(board, "X"):
                print("You win!")
                break
            if is_full(board):
                print("It's a draw!")
                break

            # AI turn
            print("AI is thinking...")
            ai_move = alpha_beta_search(board)
            if ai_move is None:
                print("AI could not find a move — it's a draw.")

```

```

        break
    board[ai_move] = "O"
    print_board(board)
    if is_winner(board, "O"):
        print("AI wins!")
        break
    if is_full(board):
        print("It's a draw!")
        break

else: # AI first
    print("AI is thinking...")
    ai_move = alpha_beta_search(board)
    if ai_move is None:
        print("AI could not find a move — it's a draw.")
        break
    board[ai_move] = "O"
    print_board(board)
    if is_winner(board, "O"):
        print("AI wins!")
        break
    if is_full(board):
        print("It's a draw!")
        break

# Human turn
move = human_move()
board[move] = "X"
print_board(board)
if is_winner(board, "X"):
    print("You win!")
    break
if is_full(board):
    print("It's a draw!")
    break

if __name__ == "__main__":
    main()

```

Alpha Beta Pruning

function AlphaBetaSearch(state) return an action $v \leftarrow \text{max-value}(state, -\infty, \infty)$ returns the action in actions(state) with value v function max-value($state, \alpha, \beta$) returns a utility value.

if Terminal - Test(state) then return utility $v \leftarrow s$

for each a in Action(state) do
 $v \leftarrow \text{max}(v, \text{min-value}(\text{Result}(s, a), \alpha, \beta))$
 if $v \geq \beta$ then return v
 $\alpha \leftarrow \max(\alpha, v)$
return v

function min-value($state, \alpha, \beta$) return a utility value
if Terminal - Test(state) then return utility(state)

$v \leftarrow -\infty$
for each a in Action(state) do:
 $v \leftarrow \min(v, \text{max-value}(\text{Result}(s, a), \alpha, \beta))$
 if $v \leq \alpha$ then return v
 $\beta \leftarrow \min(\beta, v)$
return v

Resolution

- 1) Convert all to CNF
- 2) Negate conclusions, convert result to CNF
- 3) Add negate conclusion to the premise clause
- 4) Repeat until contradiction or no progress is made.

(a) ~~select~~ select 2 clauses

(b) Resolve them together, perform all req. unification.

(c) if resolved is the empty clause a contradiction has been found.

(d) If not, add resolved to the premises.

If we succeed in step 4 we have proved the conclusion.

FOL $\rightarrow$ CNF

(1) Eliminate biconditionals, implications

(2) Move $\neg$ inwards.

(3) standard variable, apart by renaming, from.

(4) each quantifier should use different variable

(5) Skolemize: each existential variable is replaced by skolem constant or skolem function.